

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Dipartimento di Informatica

Progetto di Forza 4

Architettura Client-Server Multi-threaded

Studente: Giuseppe Pollio

Matricola: N86004953

Laboratorio di Sistemi Operativi

Docente: Prof.ssa Alessandra Rossi

Anno Accademico 2025-2026

Contents

1	Introduzione	2
2	Requisiti	2
3	Scelte architetturali	2
4	Componenti del sistema	3
4.1	Il Server	3
4.2	Il Client	3
4.3	Gestione delle Disconnessioni	3
5	Ciclo di vita di una partita	3
6	Protocollo e Implementazione	4
6.1	Struttura del messaggio	4
6.2	Tabella dei messaggi	4
6.3	Accesso concorrente sicuro alle partite	5
6.4	Elaborazione della mossa	5
6.5	Algoritmo di rilevamento vittoria	6
7	Testing	6
8	Conclusioni	7

1 Introduzione

Questo documento illustra le scelte progettuali e implementative adottate per lo sviluppo del progetto di Laboratorio di Sistemi Operativi: la realizzazione del gioco “Forza 4” in un’architettura Client-Server distribuita, in cui più giocatori possono connettersi simultaneamente, creare partite e sfidarsi in modo sincronizzato. Sia il Client che il Server sono stati sviluppati interamente in linguaggio C tramite le API POSIX.

2 Requisiti

Il server deve gestire più partite in parallelo, supportando le seguenti funzionalità:

- Accettare connessioni TCP multiple e identificare i giocatori tramite username.
- Gestire creazione, lista e join di partite, con accettazione/rifiuto da parte del creatore.
- Gestire la logica di gioco su griglia 6×7: alternanza turni, validazione mosse, rilevamento vittoria/pareggio.
- Gestire le disconnessioni improvvise in qualsiasi stato della partita, notificando l’avversario.

Tra i requisiti non funzionali: validazione sistematica di tutti gli input client, uso esclusivo di API POSIX standard, e organizzazione modulare del codice in file sorgente separati (`server.c`, `game.c`, `client_handler.c`).

3 Scelte architetturali

La comunicazione è basata su socket TCP (`AF_INET`, `SOCK_STREAM`), che garantisce affidabilità e orientamento alla connessione.

Il server è **multi-threaded**: per ogni client connesso viene creato un thread POSIX (`pthread`) in modalità *detached*. Si è preferito questo approccio rispetto ai processi (`fork`) perché `game_list` e `client_list` sono strutture globali condivise che devono essere lette e modificate da tutti i thread contemporaneamente. I thread, condividendo lo stesso spazio di indirizzamento, rendono questa comunicazione diretta ed efficiente, senza dover ricorrere a meccanismi IPC.

L’accesso concorrente a queste strutture è protetto da **mutex** (`pthread_mutex_t`), che prevengono race condition. Ogni partita ha inoltre un proprio mutex dedicato per serializzare le mosse.

Caratteristica	Thread (pthread)	Processi (fork)
Memoria condivisa	Sì, stessa heap	No, copia-su-scrittura
Overhead creazione	Basso	Elevato
Comunicazione	Diretta (mutex/condvar)	IPC (pipe, shm, socket)
Adatto a stato globale	Sì	Complesso

Per la comunicazione è stato definito un protocollo applicativo custom in `protocol.h`. Client e server si scambiano esclusivamente pacchetti a dimensione fissa (`message_t`). Le funzioni wrapper `send_all` e `recv_all` garantiscono l’assenza di letture/scritture parziali dovute ai buffer di rete.

4 Componenti del sistema

4.1 Il Server

Il Server accetta connessioni con `accept()` e crea un thread `client_handler` per ogni client. Mantiene due strutture globali:

- `client_list`: utenti attualmente connessi.
- `game_list`: partite attive, ciascuna con un `game_id` univoco, uno stato (`WAITING`, `CHALLENGING`, `PLAYING`, `FINISHED`) e un mutex dedicato.

4.2 Il Client

Il Client si connette al server, effettua il login e presenta un menu interattivo. Una volta in partita, delega il controllo a `game_loop()`, che rimane in ascolto dei messaggi del server e richiede input da tastiera solo alla ricezione di `MSG_YOUR_TURN`. Il design è volutamente *thin*: il client non mantiene alcuno stato logico, ma si limita a visualizzare quanto ricevuto dal server (*Server-Authoritative*).

4.3 Gestione delle Disconnessioni

Quando `recv_message` fallisce (≤ 0), il thread invoca `handle_disconnect_from_game`, che analizza lo stato della partita e agisce di conseguenza:

- **WAITING**: la partita viene rimossa dalla lista.
- **CHALLENGING**: se si disconnette il creatore, lo sfidante riceve `MSG_REJECTED`; se si disconnette lo sfidante, la partita torna in `WAITING`.
- **PLAYING**: l'avversario riceve `MSG_OPPONENT_LEFT` e ottiene la vittoria a tavolino.

Anche Ctrl+C è gestito tramite `sigint_handler`, che invia `MSG_QUIT` e chiude la socket pulitamente. Il segnale `SIGPIPE` è ignorato su entrambi per evitare terminazioni inattese.

5 Ciclo di vita di una partita

Una partita attraversa la seguente sequenza di stati:

$$\text{WAITING} \xrightarrow{\text{JOIN}} \text{CHALLENGING} \xrightarrow{\text{ACCEPT}} \text{PLAYING} \xrightarrow{\text{fine}} \text{FINISHED}$$

- **WAITING**: partita creata, in attesa di sfidante.
- **CHALLENGING**: sfidante connesso, in attesa di risposta dal creatore.
- **PLAYING**: partita in corso; i giocatori si alternano con `MSG_MOVE`.
- **FINISHED**: partita terminata; record rimosso e memoria liberata.

6 Protocollo e Implementazione

6.1 Struttura del messaggio

Il protocollo è basato su un unico tipo di pacchetto a dimensione fissa. Includere la griglia completa in ogni pacchetto elimina la necessità di stato lato client:

Listing 1: Struttura message_t in protocol.h

```
1 typedef struct {
2     msg_type_t type;
3     int game_id;
4     int column;
5     outcome_t outcome;
6     int board[ROWS][COLS];
7     char username[MAX_USERNAME];
8     int game_count;
9     int game_ids[20];
10    char game_creators[20][MAX_USERNAME];
11 } message_t;
```

6.2 Tabella dei messaggi

Messaggio	Direzione	Descrizione
MSG_LOGIN	C → S	Invio username
MSG_LOGIN_OK	S → C	Login confermato
MSG_CREATE_GAME	C → S	Crea nuova partita
MSG_GAME_CREATED	S → C	Partita creata (con game_id)
MSG_LIST_GAMES	C → S	Richiesta lista partite
MSG_GAME_LIST	S → C	Elenco partite in attesa
MSG_JOIN_GAME	C → S	Richiesta di unirsi a una partita
MSG_CHALLENGE	S → C	Sfida in arrivo per il creatore
MSG_ACCEPT	C → S	Creatore accetta la sfida
MSG_REJECT	C → S	Creatore rifiuta la sfida
MSG_JOIN_OK	S → C	Unione confermata
MSG_REJECTED	S → C	Sfida rifiutata
MSG_MOVE	C → S	Mossa (colonna 0-6)
MSG_BOARD_UPDATE	S → C	Griglia aggiornata
MSG_YOUR_TURN	S → C	Tocca a te
MSG_WAIT_TURN	S → C	Attendi turno avversario
MSG_GAME_OVER	S → C	Partita terminata
MSG_OPPONENT_LEFT	S → C	Avversario disconnesso
MSG_ERROR	S → C	Errore generico
MSG_QUIT	C → S	Disconnessione volontaria

6.3 Accesso concorrente sicuro alle partite

La funzione `find_and_lock_game` previene use-after-free imponendo un ordine di acquisizione dei lock: prima `game_list_lock`, poi `g->lock`, infine rilascio di `game_list_lock`. Questo garantisce che nessun thread possa liberare la partita tra la ricerca e il lock.

Listing 2: `find_and_lock_game` in `game.c`

```
1 game_t *find_and_lock_game(int game_id) {
2     pthread_mutex_lock(&game_list_lock);
3     game_t *g = game_list;
4     while (g) {
5         if (g->id == game_id) {
6             pthread_mutex_lock(&g->lock);
7             pthread_mutex_unlock(&game_list_lock);
8             return g;
9         }
10        g = g->next;
11    }
12    pthread_mutex_unlock(&game_list_lock);
13    return NULL;
14 }
```

6.4 Elaborazione della mossa

Alla ricezione di `MSG_MOVE`, il server verifica che il client sia nella partita e che sia il suo turno, quindi chiama `drop_disc`. Se la colonna è valida, aggiorna la griglia e la invia in broadcast. Controlla poi vittoria (`check_win`) o pareggio (`board_full`), invia `MSG_GAME_OVER` con l'esito corretto e rimuove la partita; altrimenti passa il turno all'avversario.

Listing 3: Gestione `MSG_MOVE` in `client_handler.c`

```
1 case MSG_MOVE: {
2     game_t *g = find_and_lock_game(msg.game_id);
3     // controlla: stato PLAYING, turno, appartenenza
4     int disc = (player_idx == 0) ? PLAYER1 : PLAYER2;
5     int row = drop_disc(g->board, msg.column, disc);
6     if (row < 0) {
7         pthread_mutex_unlock(&g->lock);
8         send_error(fd, "Colonna piena/invalida");
9         break;
10    }
11    send_board(g, 0);
12    send_board(g, 1);
13    if (check_win(g->board, disc)) {
14        res.outcome = OUTCOME_WIN;
15        send_message(g->player_fd[player_idx], &res);
16        res.outcome = OUTCOME_LOSE;
17        send_message(g->player_fd[1 - player_idx], &res);
18        pthread_mutex_unlock(&g->lock);
19        remove_game(g->id);
20    } else if (board_full(g->board)) {
21        res.outcome = OUTCOME_DRAW;
```

```

22     send_message(g->player_fd[0], &res);
23     send_message(g->player_fd[1], &res);
24     pthread_mutex_unlock(&g->lock);
25     remove_game(g->id);
26 } else {
27     g->current_turn = 1 - g->current_turn;
28     send_turn_notifications(g);
29     pthread_mutex_unlock(&g->lock);
30 }
31 break;
32 }

```

6.5 Algoritmo di rilevamento vittoria

La funzione `check_win` scansiona le quattro direzioni possibili (orizzontale, verticale, diagonale principale e secondaria) a partire da ogni cella occupata dal giocatore. Per ciascuna direzione conta i dischi adiacenti consecutivi: se raggiunge 4, restituisce vittoria. La complessità è $O(R \cdot C)$ con $R = 6$, $C = 7$, quindi costante per questa dimensione di griglia.

Listing 4: `check_win` in `game.c`

```

1  int check_win(int board[ROWS][COLS], int player) {
2      static const int dr[] = { 0, 1, 1, 1 };
3      static const int dc[] = { 1, 0, 1, -1 };
4      for (int r = 0; r < ROWS; r++) {
5          for (int c = 0; c < COLS; c++) {
6              if (board[r][c] != player) continue;
7              for (int d = 0; d < 4; d++) {
8                  int count = 1;
9                  for (int k = 1; k < 4; k++) {
10                     int nr = r + dr[d] * k;
11                     int nc = c + dc[d] * k;
12                     if (nr < 0 || nr >= ROWS || nc < 0 || nc >= COLS) break;
13                     if (board[nr][nc] != player) break;
14                     count++;
15                 }
16                 if (count == 4) return 1;
17             }
18         }
19     }
20     return 0;
21 }

```

7 Testing

Il sistema è stato testato manualmente attraverso i seguenti scenari:

Scenario	Esito atteso e verificato
Partita completa (vittoria)	Griglia aggiornata su entrambi i client; <code>MSG_GAME_OVER</code> con esiti corretti.
Pareggio	<code>board_full</code> rilevato; <code>OUTCOME_DRAW</code> inviato a entrambi.
Disconnessione durante partita	Avversario riceve <code>MSG_OPPONENT_LEFT</code> ; vittoria a tavolino.
Disconnessione in attesa	Partita rimossa dalla <code>game_list</code> senza errori.
Disconnessione in fase CHALLENGING	Sfidante: <code>MSG_REJECTED</code> . Creatore: torna in <code>WAITING</code> .
Sfida rifiutata	Sfidante riceve <code>MSG_REJECTED</code> ; creatore resta in attesa.
Colonna piena	Server risponde <code>MSG_ERROR</code> ; client può riprovare.
4 client simultanei	Nessuna interferenza tra partite indipendenti.
Username non valido	Server rifiuta la connessione con <code>send_error</code> .

Gli strumenti utilizzati durante lo sviluppo: GCC con `-Wall -Wextra` per i warning a compile time; Valgrind (`memcheck`) per la rilevazione di memory leak; GDB per il debugging di race condition nelle fasi iniziali; Git per il versionamento con commit separati per ciascuna funzionalità.

8 Conclusioni

Il progetto implementa un server reattivo e resistente agli errori in ambiente POSIX. La validazione sistematica degli input — username tramite `sanitize_username`, whitelist dei tipi di messaggio tramite `is_valid_client_msg`, numero di colonna controllato in `drop_disc` — unita al sistema di mutex e al protocollo Server-Authoritative, garantisce correttezza e robustezza anche in presenza di client malevoli o disconnessioni impreviste.

Lo spegnimento del server è progettato per non lasciare memoria non liberata né thread orfani: alla ricezione di `SIGINT` viene impostato il flag atomico `server_running = 0`, il socket principale viene chiuso per sbloccare `accept()`, e tutti i socket client vengono chiusi con `shutdown(SHUT_RDWR)` per sbloccare le `recv()` bloccanti. Il thread principale attende con `pthread_cond_wait` che `active_threads` scenda a zero, dopodiché libera `client_list` e `game_list`. Per limitare abusi, ogni client può avere al massimo `MAX_GAMES_PER_CLIENT = 5` partite attive contemporaneamente.

Possibili estensioni future includono: autenticazione persistente con database, classifica globale dei giocatori, un avversario artificiale e un'interfaccia grafica tramite ncurses.